



School of Computer Science and Artificial Intelligence

Lab Assignment # 09

Course Title : **AI Assistant Coding**
Name of Student : **BG.Sreevani**
Enrollment No. : **2303A54066**
Batch No. : **48**

Lab 9: Documentation Generation – Automatic Documentation and Code Comments

Task 1: Basic Docstring Generation

Scenario: You are developing a utility function that processes numerical lists and must be properly documented for future maintenance.

Prompt: Write a Python function to return the sum of even numbers and sum of odd numbers in a given list. Generate a Google-style Python docstring for the following function. The docstring should include: A clear description of the function's purpose, Args section with parameter names and types, Returns section with types and descriptions

Expected Output:

- Python function with manual Google-style docstring
- AI-generated docstring for the same function
- Comparison explaining differences between manual and AI-generated documentation
- Improved understanding of AI-generated function-level documentation.

Task 2: Automatic Inline Comments

Scenario:You are developing a student management module that must be easy to understand for new developers.

Prompt: You are developing a student management module that must be easy to understand for new developers.**Requirements:****Attributes:** name, roll_no, hostel_status, **Methods:** fee_update() and display_details(), Manually write inline comments for each line or logical block, Use an AI-assisted tool to automatically add inline comments, Compare manual comments with AI-generated comments, Identify missing, redundant, or incorrect AI comments

Expected Output:

- Python class with manually written inline comments
- AI-generated inline comments added to the same code
- Comparative analysis of manual vs AI comments
- Critical discussion on strengths and limitations of AI-generated comments

Output Screenshot:

```
lab9.py > ...
32
33 #You are developing a student management module that must be easy to understand for new developers.
34 #Requirements:Attributes: name, roll_no, hostel_status, Methods: fee_update() and display_details(), Manually write inline comments for each line or
35
36 class Student:
37     def __init__(self, name, roll_no, hostel_status) -> None:
38         # Initialize the Student object with provided attributes
39         self.name = name # Set the student's name
40         self.roll_no = roll_no # Set the student's roll number
41         self.hostel_status = hostel_status # Set the student's hostel status
42         self.fee = 0 # Initialize the fee to 0
43
44     def fee_update(self, amount) -> None:
45         # Update the student's fee by adding the given amount
46         self.fee += amount # Add the amount to the current fee
47
48     def display_details(self) -> None:
49         # Display the student's details
50         print(f"Name: {self.name}") # Print the student's name
51         print(f"Roll No: {self.roll_no}") # Print the student's roll number
52         print(f"Hostel Status: {self.hostel_status}") # Print the student's hostel status
53         print(f"Fee: {self.fee}") # Print the student's current fee
54
55
56
```

Task 3: Module-Level and Function-Level Documentation

Scenario: You are building a small calculator module that will be shared across multiple projects and requires structured documentation.

Prompt: You are building a small calculator module that will be shared across multiple projects and requires structured documentation. Write a Python script containing 3–4 functions (e.g., add, subtract, multiply, divide), Manually write NumPy Style docstrings for each function, Use AI assistance to generate: A module-level docstring Individual function-level docstrings, Evaluate documentation structure, accuracy, and readability

Expected Output

- Python script with manual NumPy-style docstrings
- AI-generated module-level and function-level documentation
- Comparison between AI-generated and manual documentation
- Clear understanding of structured documentation for multi-function script.

Output Screenshot:

